CS182: Introduction to Machine Learning, Fall 2025
Homework 4

(Due on Saturday, Dec. 20 at 11:59 pm (CST))

Notice:

- Please submit your assignments via Gradescope. The entry code is 428YBY.
- Please make sure you select your answer to the corresponding question when submitting your assignments.
- Each person has a total of five days to be late without penalty for all the assignments. Each late delivery less than one day will be counted as one day.

1. [20 points] [*K-means*] Given a 2D dataset $\mathcal{X}$ consisting of 5 data points:

$$x_1 = (1,1), \quad x_2 = (2,1), \quad x_3 = (4,3), \quad x_4 = (5,4), \quad x_5 = (8,5), \quad x_6 = (3,3)$$

Perform the k-Means clustering algorithm on this dataset with $k = 2$ clusters. The initial cluster centers (prototypes) are set as:

$$m_1^{(0)} = (1,1), m_2^{(0)} = (5,4)$$

(a) Calculate the Euclidean distance between each data point and the two initial centers, and assign each point to the nearest cluster. List the distance calculations for all points and the resulting cluster assignments C_1 and C_2. [4 points]

(b) Based on the cluster assignments, calculate and update the two new cluster centers $m_1^{(1)}$ and $m_2^{(1)}$. List the calculation process and the final coordinates of the new cluster centers.[6 points]

(c) The k-Means algorithm optimizes the minimization of the Total Reconstruction Error. Write down the mathematical expression for this objective function $E(\{m_i\}_{i=1}^k|\mathcal{X})$ and explain the meaning of the term b_i^t within the formula.[4 points]

(d) The Expectation-Maximization algorithm can be viewed as a probabilistic extension of k-Means. Briefly state the specific condition under which k-Means clustering is a special case of the EM algorithm applied to Gaussian Mixture Models.[6 points]

Solution:

(a)
- C_1: $\{x_1, x_2\} = \{(1,1),(2,1)\}$
- C_2: $\{x_3, x_4, x_5, x_6\} = \{(4,3),(5,4),(8,5),(3,3)\}$

(b)

$$m_1^{(1)} = \frac{(1,1)+(2,1)}{2} = \left(\frac{1+2}{2}, \frac{1+1}{2}\right) = (1.5, 1)$$

$$m_2^{(1)} = \frac{(4,3)+(5,4)+(8,5)+(3,3)}{4} = \left(\frac{4+5+8+3}{4}, \frac{3+4+5+3}{4}\right) = \left(\frac{20}{4}, \frac{15}{4}\right) = (5, 3.75)$$

Answer:

$$m_1^{(1)} = (1.5, 1), \quad m_2^{(1)} = (5, 3.75)$$

(c) The mathematical expression is:

$$E(\{m_i\}_{i=1}^k|\mathcal{X}) = \sum_t \sum_i b_i^t ||x^t - m_i||^2$$

The term b_i^t is an indicator variable (or hard label) for the data point x^t. It is defined as:

$$b_i^t = \begin{cases} 1 & \text{if } i = \arg\min_j ||x^t - m_j|| \\ 0 & \text{otherwise} \end{cases}$$

It equals 1 if x^t is assigned to the cluster represented by center m_i (i.e., m_i is the closest center to x^t), and 0 otherwise.

(d) The required simplifying conditions are:

- Inputs are assumed independent with equal and shared variances ($\Sigma_i = \sigma^2 I$).
- All components have equal priors ($P(\mathcal{G}_i)$).
- Labels are hardened (i.e., the soft posterior probabilities from the E-step are replaced by hard assignments based on the closest center).

2. [20 points] [*K-Means*] *K-Means* is a classic unsupervised clustering algorithm whose goal is to partition data into k clusters such that each sample is assigned to the cluster with the nearest center. In 'problem2.py', a KMeans class mimicking the sklearn interface is provided. You are required to complete the 'fit' method, implement the full clustering functionality, and apply it to the Iris dataset.

   (a) Complete the 'fit()' method in the given 'KMeans' class and implement a full K-Means clustering algorithm. Your answer should include: a comment on your code and a visual screenshot. (Implement the algorithm by hand and don't use the sklearn implementation.) [10 points]

   (b) After applying your 'KMeans' implementation to the Iris dataset (after reducing the data to two dimensions using PCA), you should output the resulting cluster centers ('cluster_centers_') along with the clustering visualization figure generated by your program. [5 points]

   (c) Given two new 2D samples:

$$x_1 = [1.0,\ -0.5], \quad x_2 = [2.2,\ 1.1]$$

   Use the 'predict()' method to determine whether they are assigned to the same cluster. [5 points]

Solution:

(a) Code:

```python
import numpy as np

class KMeans:
    def __init__(self, n_clusters=3, max_iter=300, tol=1e-4, random_state=None):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.tol = tol
        self.random_state = random_state

    def fit(self, X):
        if self.random_state is not None:
            np.random.seed(self.random_state)

        # TODO: Implement the KMeans clustering algorithm.
        # Note: You should have attributes `cluster_centers_` and `labels_` after fitting.
        # ============================================================
        N = X.shape[0]

        # Initialize cluster centers by selecting random samples
        idx = np.random.choice(N, self.n_clusters, replace=False)
        self.cluster_centers_ = X[idx]

        for _ in range(self.max_iter):
            # Compute distances between all samples and all centers
            distances = np.linalg.norm(
                X[:, None] - self.cluster_centers_[None, :], axis=2
            )

            # Assign each point to the nearest cluster
            labels = np.argmin(distances, axis=1)

            # Update cluster centers by computing the mean of each cluster
            new_centers = np.array([
                X[labels == j].mean(axis=0) for j in range(self.n_clusters)
            ])

            # Check convergence (center shift smaller than tolerance)
            shift = np.linalg.norm(new_centers - self.cluster_centers_)
            if shift < self.tol:
                break

            self.cluster_centers_ = new_centers

        self.labels_ = labels
        # ============================================================
        return self

    def predict(self, X):
        # Compute distances and assign cluster labels
        distances = np.linalg.norm(
            X[:, None] - self.cluster_centers_[None, :], axis=2
        )
```

```
        return np.argmin(distances, axis=1)

    def fit_predict(self, X):
        self.fit(X)
        return self.labels_


if __name__ == '__main__':
    import matplotlib.pyplot as plt
    from sklearn.datasets import load_iris
    from sklearn.decomposition import PCA

    # Load Iris dataset
    iris = load_iris()
    X = iris.data

    # Reduce to 2D using PCA for visualization
    pca = PCA(n_components=2)
    X_2d = pca.fit_transform(X)

    # Apply custom KMeans
    kmeans = KMeans(n_clusters=3, random_state=42)
    kmeans.fit(X_2d)
    labels = kmeans.labels_
    centers = kmeans.cluster_centers_
    print('\nCluster Centers:\n', centers)

    # Create a mesh grid for decision boundary visualization
    x_min, x_max = X_2d[:, 0].min() - 1, X_2d[:, 0].max() + 1
    y_min, y_max = X_2d[:, 1].min() - 1, X_2d[:, 1].max() + 1

    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
                         np.linspace(y_min, y_max, 500))

    # Predict cluster labels for all grid points
    Z = kmeans.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot decision boundary
    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, cmap='viridis', alpha=0.4)

    # Plot data points
    plt.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='viridis', s=30)

    # Plot cluster centers
    plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', s=200)

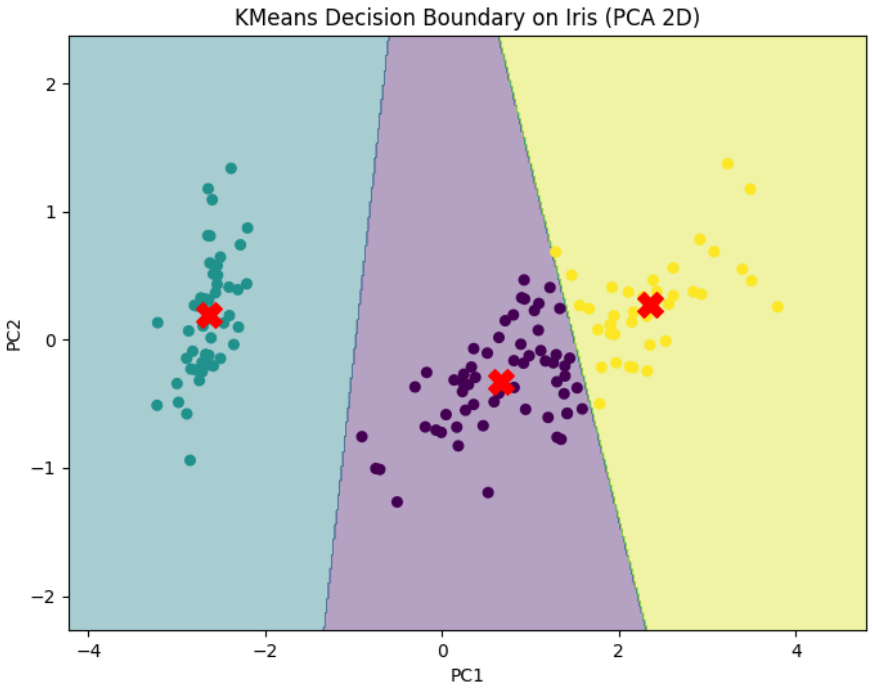
    plt.title('KMeans Decision Boundary on Iris (PCA 2D)')
    plt.xlabel('PC1')
    plt.ylabel('PC2')
    plt.show()
```

(b) The cluster centers are

$$[[0.66567601, -0.3316042], [-2.64241546, 0.19088505], [2.34652659, 0.27393856]]$$

The result is shown in the following figure 2:

(c) x_1 and x_2 belong to different clusters.



3. [20 points] [*EM*]

Let Z represent the unobserved assignment of a given observation to one of the k clusters:

$$Z \sim \text{Categorical}(\pi_1, \cdots, \pi_k)$$

where π_k is the probability that a randomly selected observation is assigned to cluster k.
Let $(X_1, Z_1), \cdots, (X_n, Z_n)$ denote the set of observations and their corresponding cluster assignments, under i.i.d. assumptions. Consider the marginalized log-likelihood function

$$l_{\text{margin}}(\theta) = \sum_{i=1}^{n} \log p(X_i; \theta) = \sum_{i=1}^{n} \log \sum_{j=1}^{k} p(X_i, Z_i = j; \theta)$$

where θ is some parameter we want to estimate. Instead of maximizing the marginalized log-likelihood function, now the EM algorithm aims to maximize a lower bound $\mathcal{F}(q, \theta)$ on the marginalized log-likelihood function, such that

$$l_{\text{margin}}(\theta) \geq \mathcal{F}(q, \theta) = \sum_{i=1}^{n} \mathcal{F}_i(q_i, \theta) \tag{1}$$

where $\mathcal{F}_i(q_i, \theta) := \sum_{z=1}^{k} q_i(z) \log \frac{p(X_i, Z_i = z; \theta)}{q_i(z)}$. Here q_i can be seen as an arbitrary distribution over the k clusters for the i-th observation.

(a) Show the above inequality holds for any $q = (q_1, \cdots, q_n)$, i.e.

$$l_{\text{margin}}(\theta) \geq \sum_{i=1}^{n} \sum_{z=1}^{k} q_i(z) \log \frac{p(X_i, Z_i = z; \theta)}{q_i(z)}$$

[6 points]

(b) For a fixed θ, the EM algorithm aims to optimize over the distributions $q_1, \cdots, q_n$, in order to make $\mathcal{F}(q, \theta)$ as close as possible to $l_{\text{margin}}(\theta)$. The following is the iterative update at iteration t of the algorithm:

$$\begin{cases} q_i^{(t+1)} = \arg\max_{q_i} \mathcal{F}(q_i^{(t)}, \theta^{(t)}) & \text{(E-step)} \\ \theta^{(t+1)} = \arg\max_{\theta} \mathcal{F}(q^{(t+1)}, \theta^{(t)}) & \text{(M-step)} \end{cases}$$

Show that for any fixed θ and $i \leq n, z \leq k$ when $q_i(z) = p(Z_i = z|X_i; \theta)$,

$$l_{\text{margin}}(\theta) = \sum_{i=1}^{n} \mathcal{F}_i(q_i, \theta)$$

i.e. the bound is sharp and $p(Z_i|X_i; \theta)$ is an optimal choice for q_i. [7 points]

(c) Show this algorithm converge, you can show this by showing $l_{\text{margin}}(\theta^{(t)}) \leq l_{\text{margin}}(\theta^{(t+1)})$ when choosing $q_i^{(t+1)}(z) = p(Z_i = z|X_i; \theta^{(t)})$. [7 points]

Solution:

(a) *Proof.* By the law of total probability

$$\log p(X_i; \theta) = \log \sum_{z=1}^{K} p(X_i, Z = z; \theta).$$

Let q_i be an arbitrary distribution over cluster assignments $1, \ldots, k$. Assuming all $q_i(z) > 0$, we have that

$$\log \sum_{z=1}^{K} p(X_i, Z = z; \theta) = \log \sum_{z=1}^{K} q_i(z) \frac{p(X_i, Z = z; \theta)}{q_i(z)},$$

and by application of Jensen's inequality,

$$\log \sum_{z=1}^{K} p(X_i, Z = z; \theta) \geq \sum_{z=1}^{K} q_i(z) \log \frac{p(X_i, Z = z; \theta)}{q_i(z)}.$$

Summing these inequalities across all data points completes the proof. □

(b) Plugging in $q_i = p(Z_i \mid X_i; \theta)$ into $\mathcal{F}_i(q_i, \theta)$ yields

$$\begin{aligned}
\mathcal{F}_i(p(Z_i \mid X_i; \theta), \theta) &= \sum_{z=1}^{k} p(Z_i = z \mid X_i; \theta) \log \frac{p(X_i, Z_i = z; \theta)}{p(Z_i = z \mid X_i; \theta)} \\
&= \sum_{z=1}^{K} p(Z_i = z \mid X_i; \theta) \log p(X_i; \theta) = \log p(X_i; \theta),
\end{aligned}$$

Summing over all i, we get that $l_{\text{marginal}}(\theta) = \sum_{i=1}^{n} \mathcal{F}_i(q_i, \theta)$.

(c) *Proof.*

$$\begin{aligned}
l_{\text{margin}}(\theta^{(t+1)}) &\geq \sum_{i=1}^{n} \mathcal{F}_i(q_i^{(t+1)}, \theta^{(t+1)}) \\
&\geq \sum_{i=1}^{n} \mathcal{F}_i(q_i^{(t+1)}, \theta^{(t)}) \\
&= l_{\text{margin}}(\theta^{(t)})
\end{aligned}$$

The first inequality holds since (1) holds for all θ, q and the second follows from that $\theta^{(t+1)}$ is chosen explicitly to be $\arg\max_\theta \mathcal{F}(q^{(t+1)}, \theta^{(t)})$ □

4. [20 points] [*Nonparametric Density Estimation*] Let us consider kernel density estimation with a boxcar kernel. Given data $X_1, \ldots, X_n$, the empirical pdf is defined as follows

$$\hat{p}_h(x) = \frac{1}{n} \frac{\sum_{i=1}^{n} I(|x - X_i| \leq h)}{2h}.$$

Here, $|y|$ indicates absolute value and $I(y)$ is an indicator function that is 1 if the boolean argument evaluates to true and 0 otherwise.

(a) Draw $\hat{p}_h(x)$ for all $x \in [-7, 7]$ where our data consists of the following observations $\{X_i\} = \{-7, -4, -3, -2, 1, 5\}$ and $h = 3$. [6 points]

(b) Let $F(x) = P(X \leq x)$ represent the true cdf over a random variable X. Show that

$$E(\hat{p}_h(x)) = \frac{F(x+h) - F(x-h)}{2h}.$$

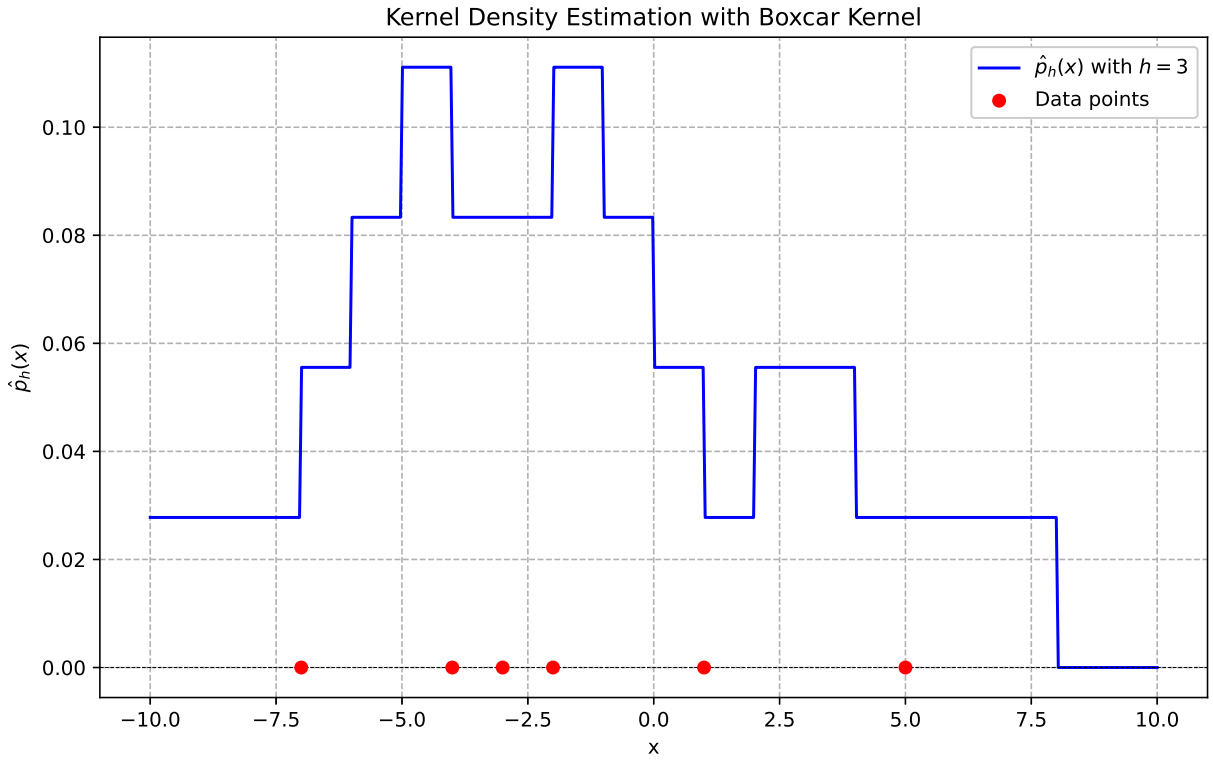
[8 points]

(c) Determine if the kernel density estimator in this question will be an unbiased estimator in the following scenarios (and you need to give an explanation):

- 1. $\text{pdf}(x)$ is uniform between 0 and 1. $\hat{p}_h\left(\frac{1}{2}\right)$ is estimated using $h = \frac{1}{3}$.
- 2. $\text{pdf}(x) = 2x$ for $0 \leq x \leq 1$. $\hat{p}_h\left(\frac{1}{4}\right)$ is estimated using $h = \frac{1}{5}$.
- 3. $\text{pdf}(x) = \frac{3}{2}x^2$ for $-1 \leq x \leq 1$. $\hat{p}_h(0)$ is estimated using $h = \frac{1}{5}$.[6 points]

Solution:

(a) The result is like the following figure:



(b) We have

$$\begin{aligned}
\mathbb{E}(\hat{p}_h(x)) =& \mathbb{E}\left(\frac{1}{n}\frac{\sum_{t=1}^{n} I(|x - x^t| \leq h)}{2h}\right) \\
=& \frac{1}{2hn}\mathbb{E}\left(\sum_{t=1}^{n} I(|x - x^t| \leq h)\right) \\
=& \frac{1}{2hn}\sum_{t=1}^{n}\mathbb{E}(I(|x - x^t| \leq h)).
\end{aligned}$$

Here note that $\mathbb{E}\left(I\left(|x-x^t|\right)\right)$ is the same for all t. Therefore I can use just X instead of x^t, thus

$$\begin{aligned}\mathbb{E}\left(\hat{p}_h\left(x\right)\right) =& \frac{1}{2h}\mathbb{E}\left(I\left(|x-X|\leq h\right)\right)\\ =& \frac{1}{2h}P\left(|x-X|\leq h\right)\\ =& \frac{1}{2h}P\left(-h\leq x-X\leq h\right)\\ =& \frac{F\left(x+h\right)-F\left(x-h\right)}{2h}.\end{aligned}$$

Note that $\text{pdf}\left(x\right)=\frac{dF(x)}{dx}$. From part two, we see that the expected pdf that we get from the boxcar kernel is just a linear approximation of the gradient of $F\left(x\right)$. So intuitively, for cases 1 and 2, the estimator will be unbiased since linear approximation of a constant or linear pdf is going to be accurate, but for case 3, the linear approximation of a quadratic pdf is not going to be without bias.

(c) 1. True. For a uniform pdf, $F\left(x\right)=0$ for $x<0$, $F\left(x\right)=x$ for $0\leq x\leq 1$ and $F\left(x\right)=1$ for $x>1$. From part two, bias of the estimator is $\left(F\left(\frac{5}{6}\right)-F\left(\frac{1}{6}\right)\right)/\left(\frac{2}{3}\right)=1=\text{pdf}\left(\frac{1}{2}\right)$.
2. True. For this case, $F\left(x\right)=0$ for $x<0$, $F\left(x\right)=x^2$ for $0\leq x\leq 1$ and $F\left(x\right)=1$ for $x>1$. From part two, bias of the estimator is $\left(F\left(\frac{9}{20}\right)-F\left(\frac{1}{20}\right)\right)/\left(\frac{2}{5}\right)=\frac{1}{2}=\text{pdf}\left(1/4\right)$.
3. False. For this case, $F\left(x\right)=0$ for $x<-1$, $F\left(x\right)=\frac{x^3+1}{2}$ for $-1\leq x\leq 1$ and $F\left(x\right)=1$ for $x>1$. From part two, bias of the estimator is $\left(F\left(\frac{1}{5}\right)-F\left(-\frac{1}{5}\right)\right)/\left(\frac{2}{5}\right)\neq 0=\text{pdf}\left(0\right)$.

5. [20 points] [*Nonparametric Regression*] We observe an i.i.d. univariate data set

$$\{(x_i, y_i)\}_{i=1}^n, \qquad x_i \in [0,1],\ y_i \in \mathbb{R},$$

generated from the non-parametric regression model

$$y_i = g(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where the unknown regression function $g\colon [0,1] \to \mathbb{R}$ is twice continuously differentiable. No parametric form for g is assumed. We estimate g with the kernel smoother

$$\hat{g}_h(x) = \frac{\sum_{j=1}^n K\left(\frac{x - x_j}{h}\right) y_j}{\sum_{j=1}^n K\left(\frac{x - x_j}{h}\right)}, \qquad K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

(a) [4 points] Show that $\hat{g}_h(x)$ is the minimiser of the *local* weighted least-squares criterion

$$\min_{\beta \in \mathbb{R}} \sum_{j=1}^n K\left(\frac{x - x_j}{h}\right)(y_j - \beta)^2.$$

(b) [8 points] Derive the conditional bias

$$\mathrm{Bias}\big[\hat{g}_h(x) \mid x_1, \ldots, x_n\big] = \mathbb{E}\big[\hat{g}_h(x) \mid \{x_j\}\big] - g(x)$$

up to order $O(h^2)$.

Hints: Use the fact: the term $\sum_j w_j(x)(x_j - x)$ is approximately zero because the kernel K is symmetric and, for small h, the weights are concentrated around x. And refer to the following process for derivation.

1. Start from the definition of $\mathbb{E}[\hat{g}_h(x) \mid \{x_j\}]$, use the model $y_j = g(x_j) + \varepsilon_j$ and properties of ε_j to simplify.
2. For $g(x_j)$, use a second-order Taylor expansion around x: $g(x_j) = g(x) + g'(x)(x_j - x) + \frac{1}{2}g''(x)(x_j - x)^2 + o((x_j - x)^2)$. Substitute and simplify.
3. You need the following regularity conditions: g is twice continuously differentiable in a neighborhood of x, and its derivatives $g'(x)$ and $g''(x)$ are bounded.
4. Your final result should be of the form $\mathrm{Bias}[\hat{g}_h(x) \mid \{x_j\}] = C(x) \cdot h^2 + o(h^2)$. Explicitly give the coefficient $C(x)$ and explain its meaning (e.g., how does it relate to the properties of g?).

(c) [8 points] Programming task: Visualising the effect of h on kernel regression. In this task, you will implement a kernel regression smoother in Python and observe how the h affects the estimate. Please follow the steps below to complete.

1. Generate $n = 200$ data points as follows: Let x_i be uniformly distributed on $[0,1]$. Define the true function $g(x) = \sin(4\pi x)$. Generate $y_i = g(x_i) + \varepsilon_i$, where $\varepsilon_i \sim \mathcal{N}(0, 0.1^2)$.
2. Implement the kernel smoother $\hat{g}_h(x)$ with the Gaussian kernel $K(u) = \frac{1}{\sqrt{2\pi}} \exp(-u^2/2)$. Write a function that takes as input the data $\{(x_i, y_i)\}$, a h, and a set of query points (e.g., 100 equally spaced points in $[0,1]$), and returns the kernel regression estimates at the query points.
3. For each of the following h: $h = 0.01, 0.05, 0.1, 0.3, 0.5, 1.0$, compute the kernel regression estimate on 100 equally spaced points in $[0,1]$. Plot the resulting estimates on the same figure, along with the true function $g(x) = \sin(4\pi x)$ and the generated data points. Use a different color for each h and include a legend.

Describe what happens as h increases. Which h seems to give the best fit? (**Notes:** You don't have to submit the code on the assignment, but you must present the drawn results in the assignment.)

Solution:

(a) Define

$$\ell(\beta) = \sum_{j=1}^n w_j (y_j - \beta)^2, \qquad w_j = K\left(\frac{x - x_j}{h}\right).$$

Differentiate:

$$\frac{\mathrm{d}\ell}{\mathrm{d}\beta} = -2\sum_{j=1}^n w_j(y_j - \beta) = 0 \implies \beta = \frac{\sum w_j y_j}{\sum w_j} = \hat{g}_h(x).$$

Second derivative is $2\sum w_j > 0$, hence a minimum.

(b) Write

$$\mathbb{E}[\hat{g}_h(x) \mid \{x_j\}] = \frac{\sum w_j g(x_j)}{\sum w_j}.$$

Taylor-expand $g(x_j)$ around the evaluation point x:

$$g(x_j) = g(x) + g'(x)(x_j - x) + \frac{1}{2}g''(x)(x_j - x)^2 + R_j, \qquad |R_j| \le C|x_j - x|^3.$$

Insert:

$$\mathbb{E}[\hat{g}_h(x) \mid \{x_j\}] - g(x) = \frac{\sum w_j \left[g'(x)(x_j - x) + \frac{1}{2}g''(x)(x_j - x)^2 + R_j\right]}{\sum w_j}.$$

Symmetry of the Gaussian kernel implies $\sum w_j(x_j - x) \approx 0$. Hence the leading term is

$$\frac{1}{2}g''(x)\frac{\sum w_j(x_j - x)^2}{\sum w_j}.$$

For interior x and $h \to 0$, standard kernel calculations give

$$\frac{\sum w_j(x_j - x)^2}{\sum w_j} = h^2 \int u^2 K(u)\, du + o(h^2) = h^2 + o(h^2)$$

Therefore

$$\text{Bias}[\hat{g}_h(x) \mid \{x_j\}] = \frac{1}{2}g''(x)h^2 + o(h^2).$$

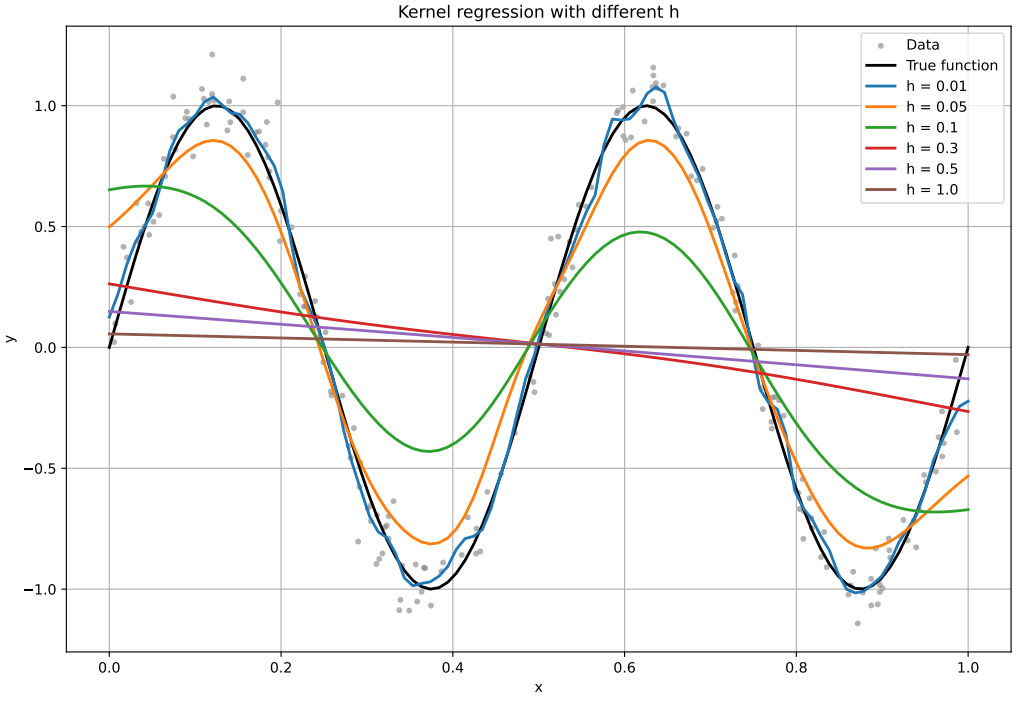


Figure 1: Kernel regression estimates for different h. The gray dots are the generated data, the black curve is the true function $g(x) = \sin(4\pi x)$, and the colored curves are the kernel regression estimates.

(c)
1. **Small h (e.g., 0.01)**: The estimate is very wiggly and closely follows the individual data points. This corresponds to high variance (the curve changes dramatically with small changes in the data) and low bias (the curve passes near each data point).
2. **Moderate h (e.g., 0.05, 0.1)**: The estimate is smoother and captures the main oscillations of the true function without too much noise.
3. **Large h (e.g., 0.3, 0.5, 1.0)**: The estimate becomes very smooth, essentially a flat line for $h = 1.0$. This corresponds to low variance (the curve is insensitive to the data) but high bias (the curve cannot capture the oscillations of the true function).